

Calibrated Uncertainty Quantification for Physics-Informed Neural Networks via Learned Multi-Feature Conformal Scores

A complete step-by-step research plan and execution guide

for an MTech student under Dr. Nanda Dulal Jana with mentor Sayan, Department of CSE, NIT Durgapur

Document overview. This document is a single, self-contained guide for executing a research project on uncertainty quantification (UQ) for Physics-Informed Neural Networks (PINNs). It contains: (i) the research hypothesis stated formally, (ii) the theoretical framing, (iii) a literature review identifying the gap, (iv) a complete experimental protocol with PDE benchmarks and baselines, (v) runnable code skeletons for every component, (vi) a phase-by-phase execution timeline with weekly todos, (vii) writing guidance and target venues, (viii) reading list, (ix) a risk register and troubleshooting reference. It is designed so that an MTech student can execute the project end-to-end with periodic mentor checkpoints, producing a publishable manuscript at the end.

Contents

1. Project at a Glance
2. Background and Motivation
3. Literature Review and the Gap
4. Research Hypothesis and Falsification Criteria
5. Theoretical Framing
6. Methodology
7. Models and Architectures
8. Code Organization
9. Experimental Protocol
10. PDE Benchmarks
11. Baselines
12. Metrics and Evaluation
13. Phase-by-Phase Execution Plan
14. Weekly Todos
15. Writing the Paper
16. Risk Register
17. Troubleshooting Reference
18. Reading List
19. Mentor Checkpoints and Communication Appendix A. Default Hyperparameters Appendix B. Code Skeletons Appendix C. References

1. Project at a Glance

PINNs are neural networks trained to satisfy a partial differential equation (PDE) plus initial and boundary conditions. They give us a function $u_\theta(x)$ approximating the true solution $u^*(x)$. The problem this project addresses: **PINNs do not tell you how wrong they are.** We want to attach calibrated error bars to PINN predictions, with rigorous statistical guarantees, especially in the regions where existing UQ methods fail — namely, near shocks and sharp interfaces.

The methodology is built on two ingredients. The first is split conformal prediction (CP), a distribution-free framework that turns any heuristic uncertainty score into a calibrated prediction interval with finite-sample coverage guarantees. The second is a small learned MLP that combines several physics-derived features of the PINN's prediction (PDE residual, gradient norm, second-derivative norms) into a single nonconformity score. The combination is novel: existing CP-for-PINN papers use single hand-picked scores, and existing learned-CP papers do not use physics features. The empirical claim is that the learned multi-feature score yields tighter and better-calibrated intervals than the residual baseline, with the largest gains on PDEs with shocks.

1.1 One-paragraph contribution

We propose a learned, multi-feature, physics-derived nonconformity score for split conformal prediction on PINNs. On singularity-bearing PDE benchmarks (Burgers, Buckley–Leverett, Allen–Cahn) the resulting prediction intervals achieve target marginal coverage by construction and substantially better near-shock conditional coverage than residual-based, MC-dropout-based, or ensemble-based conformal baselines, with comparable or smaller mean width. The wrapper is model-agnostic: it can be applied to any pre-trained PINN.

1.2 What success looks like

- On Burgers ($\nu = 0.01/\pi$) the learned score achieves near-shock conditional coverage within 5 percentage points of target (90 %), where the residual baseline undercovers by 10 percentage points.
- Mean interval width is at most that of the residual baseline globally, and at most $1.2\times$ near the shock.
- The pattern replicates on Buckley–Leverett and Allen–Cahn at five PINN seeds each.
- A complete, reviewable manuscript is prepared and submitted to a target venue (Neural Networks, JCP, or TMLR).

1.3 What failure looks like

- If after a fully converged PINN (PDE loss $\leq 1e-3$ with L-BFGS finetuning) the learned score fails to beat the residual baseline on near-shock conditional coverage by at least 5 percentage points across 5 seeds, the trunk hypothesis is falsified.
- If the win on Burgers does not generalize to *any* of the other two PDEs, the contribution is too narrow and the project should pivot.
- Either failure is itself publishable as a careful negative result, but it is not the planned outcome.

2. Background and Motivation

2.1 PINNs in one paragraph

A PINN (Raissi, Perdikaris, Karniadakis 2019) is a feedforward neural network $u_\theta: \Omega \rightarrow \mathbb{R}$ trained to minimize the sum of three losses: (i) the PDE residual $L_{\text{PDE}} = \mathbb{E}[(F(u_\theta))^2]$ at randomly sampled collocation points, (ii) the initial-condition mismatch, (iii) the boundary-condition mismatch. The PDE residual is computed via automatic differentiation through the network. After training, u_θ approximates u^* throughout Ω , in principle without ever seeing labeled data inside Ω .

2.2 Why PINN UQ is hard

Three structural issues. First, PINNs do not have access to true u^* during training, so standard supervised-learning UQ tools (predictive variance, calibration loss) do not apply directly. Second, the PDE residual is the **only** training signal in the interior of Ω , but residual $\neq$ error: a PINN can have small residual at a smoothed shock while having large error there. Third, the most active failure mode of PINNs is at solution singularities, where the strong-form residual loses signal exactly when uncertainty is most needed.

2.3 Existing PINN UQ methods

- **Bayesian PINNs (B-PINN)** using HMC or variational inference (Yang, Meng, Karniadakis 2021). Posterior variance as uncertainty. Heavy compute, prior-dependent, no finite-sample coverage guarantee.
- **Monte Carlo dropout PINNs**. Cheap; widely used; well-known to be poorly calibrated for regression.
- **Deep ensembles**. Train k PINNs with different seeds; report variance. Strong baseline but $k \times$ compute.
- **Evidential deep learning variants** (Sensoy et al., Amini et al.). Newer; not yet standard in PINN context.
- **Physics-informed GAN posteriors** (Yang & Perdikaris 2018). Niche; unstable training.

None of the above provide distribution-free finite-sample coverage guarantees. They are heuristic. The recent shift in the literature has been to wrap these heuristics in conformal prediction (CP) to obtain rigorous coverage.

2.4 Conformal prediction in one paragraph

Split CP (Vovk, Gammerman, Shafer 2005; Lei et al. 2018): hold out a calibration set $D_{\text{cal}} = \{(x_i, y_i)\}_{i=1}^n$. Pick any nonconformity score function $s(x) > 0$ computable from the model alone. Compute scaled residuals $r_i = |y_i - u_\theta(x_i)| / s(x_i)$, take the $\lceil (n+1)(1-\alpha) \rceil / n$ -th order statistic q . For a new test point output the interval $[u_\theta(x) \pm q \cdot s(x)]$. By exchangeability, $P(y_{\text{test}} \in \text{interval}) \geq 1 - \alpha$, finite-sample, distribution-free, for **any** choice of s . The choice of s controls how the marginal coverage budget is distributed across the input space.

3. Literature Review and the Gap

Closely related work, in order of proximity to the project:

Paper	What they do	Limitation we exploit
Yu, Ho, Wang 2025 (arXiv 2509.13717)	First CP framework for PINNs. Wraps MC dropout, B-PINN posterior variance, distance-based UQ in split CP. Adds a localized conformal quantile estimator (LCQE) for spatial heteroskedasticity.	Tests only on damped oscillator, Poisson, Allen–Cahn, Helmholtz — none with shocks . Each score is hand-picked, not learned. Single-feature.
Gopakumar et al. 2025 (arXiv 2502.04406)	Calibrated Physics-Informed UQ. Uses raw PDE residual as the CP score for neural-PDE surrogates. Marginal and joint CP variants.	Single-feature (residual only). No learning. Surrogate-model setting differs from PINN.
Podina et al. 2024 (C-PINN)	Earlier conformal-PINN, including Buckley–Leverett. Residual-based CP score.	Single feature; older methodology.
Kumar et al. 2025 (arXiv 2509.21955)	Learnable Conformal Prediction with context-aware nonconformity functions. MLP nonconformity score $s_\theta(x) = f_\theta(\varphi(x))$ for robotics.	Not for PINNs . Features are semantic / image-geometric, not physics-derived. Establishes that learned CP scores are valid in principle.
Harris & Liu 2025 (LSCI, arXiv 2507.20975)	Locally-adaptive CP for neural operators with generic distance-based localizers.	Operator-learning setting (different from single-instance PINN). Generic localizers, not physics-aware.
Romano et al. 2019 (CQR)	Conformalized Quantile Regression — fit conditional quantile estimators, conformalize.	Generic regression. Not physics-informed.
Foygel-Barber et al. 2021	Theoretical limits of distribution-free conditional CP coverage.	Important to cite — bounds what we can claim about <i>conditional</i> coverage.

3.1 The gap, stated precisely

No published work combines (i) learned multi-feature nonconformity scores, (ii) physics-derived features computed from the PINN itself, and (iii) singularity-bearing PDE benchmarks where residual-based scores are known to fail. The intersection of these three is the contribution of this project.

4. Research Hypothesis and Falsification Criteria

4.1 Trunk hypothesis

H1. Given a converged PINN u_θ for a PDE $F[u] = 0$ on Ω , a calibration set on which u^* is known, and a feature map $\varphi: \Omega \rightarrow \mathbb{R}^k$ computable from u_θ at x alone (residual, derivative norms), a small MLP g_φ trained on a sub-split of the calibration set to predict $\log|u_\theta(x) - u^*(x)|$ from $\varphi(x)$, used as the nonconformity score $s(x) = \exp(g_\varphi(\varphi(x)))$ in split CP, yields:

- (a) marginal coverage at the prescribed level (guaranteed by CP exchangeability);
- (b) strictly better near-shock conditional coverage than residual-based CP, by at least 5 percentage points;
- (c) mean interval width comparable to or smaller than residual-based CP globally, and at most 1.2× larger near shocks.

4.2 Subsidiary hypotheses

- H2. The win on Burgers replicates qualitatively on Buckley–Leverett and Allen–Cahn.
- H3. The pattern is robust across at least 5 PINN training seeds.
- H4. The minimum useful feature set is < 5 features; richer features do not significantly help.
- H5. The minimum useful MLP is small (≤ 2 hidden layers, ≤ 64 width); deeper MLPs do not help.

4.3 Falsification criteria

The project is willing to die against any of these:

1. If on a converged Burgers PINN (PDE loss $\leq 1e-3$) the learned MLP score fails to beat residual-CP on near-shock conditional coverage by at least 5 pp across 5 seeds, H1 is falsified.
2. If H1 holds on Burgers but fails entirely on both BL and AC, H2 is falsified and the contribution is too narrow.
3. If the learned MLP requires more than 30 % of the calibration budget to fit (i.e., $n_{\text{train}} > 30\%$ of n_{cal}), the framework is impractical relative to single-feature baselines.

Pilot-experiment evidence: Three preliminary pilots have been run by the mentor. On Burgers $v=0.01/\pi$ at low convergence (PDE loss ~ 0.18), the learned MLP achieved $\text{cov}_n = 0.97$ vs target 0.90, beating residual-CP at $\text{cov}_n = 0.81$. Width dropped 53 % globally and 11 % near the shock. A cross-seed test showed instability at low convergence (one of two cross-seeds inverted the pattern); the project plan therefore frames the wrapper as *per-PINN*, with full-convergence cross-seed transfer as a Phase-7 ablation rather than a load-bearing claim.

5. Theoretical Framing

5.1 Notation

- Domain $\Omega \subset \mathbb{R}^d \times [0, T]$ for time-dependent problems; $\Omega \subset \mathbb{R}^d$ for stationary.
- PDE $Fu = 0$ with appropriate IC/BC. Examples: $F[u] = u_t + u u_x - \nu u_{xx}$ (Burgers).
- Trained PINN $u_\theta: \Omega \rightarrow \mathbb{R}$. Reference solution $u^*: \Omega \rightarrow \mathbb{R}$ from a high-fidelity numerical method.
- Residual $R_\theta(x) = F u_\theta$ (autograd through the network).
- Feature map $\varphi: \Omega \rightarrow \mathbb{R}^k$, computable from u_θ alone.
- Nonconformity score $s: \Omega \rightarrow \mathbb{R}_{>0}$; standard split CP procedure.
- Target miscoverage $\alpha \in (0, 1)$; typical $\alpha = 0.1$.

5.2 The proposition the paper proves

Proposition (split-CP coverage with learned MLP score). Let $(x_1, y_1), \dots, (x_{n+1}, y_{n+1})$ be exchangeable random variables with $y_i = u^*(x_i)$. Let u_θ be a fixed trained PINN. Let $\varphi: \Omega \rightarrow \mathbb{R}^k$ be a measurable feature map, and let $g_\varphi: \mathbb{R}^k \rightarrow \mathbb{R}$ be a measurable function fit on a subset of $\{1, \dots, n\}$ of size m , leaving the conformal calibration set $D_{\text{conf}} = \{(x_i, y_i) : i \in I_{\text{conf}}\}$, $|I_{\text{conf}}| = n - m$, disjoint from the MLP training subset. Define $s(x) = \exp(g_\varphi(\varphi(x)))$ and the standard split-CP quantile q . Then for any $\alpha \in (0, 1)$,

$$\mathbb{P}(y_{n+1} \in [u_\theta(x_{n+1}) - q \cdot s(x_{n+1}), u_\theta(x_{n+1}) + q \cdot s(x_{n+1})]) \geq 1 - \alpha.$$

Proof sketch. Standard exchangeability argument (Vovk et al. 2005; Lei et al. 2018). The MLP g_φ is a deterministic function once fit. Because g_φ is fit on a subset disjoint from D_{conf} , conditional on g_φ , the conformal scores $r_i = |y_i - u_\theta(x_i)| / s(x_i)$ for $i \in I_{\text{conf}} \cup \{n+1\}$ are exchangeable. The empirical $(1-\alpha)$ -quantile q then has the standard rank guarantee, which gives the coverage statement. ■

Important caveat: The proposition guarantees *marginal* coverage only. Distribution-free *conditional* coverage is impossible in general (Foygel-Barber et al. 2021). We report conditional coverage (near-shock, far) as an empirical diagnostic — a region where our method should empirically improve over residual-CP — not as a theoretical guarantee.

5.3 Why the learned multi-feature score helps near shocks

Heuristic argument. Near a shock the PINN smooths the discontinuity. The strong-form residual is locally near-zero (the smoothed solution satisfies the PDE pointwise) but the true error is large. Single-feature residual-based CP therefore systematically *underweights* the shock region in its quantile, leading to

undercoverage there. The gradient norm $\|\nabla u_\theta\|$ is large at the smoothed shock and partially compensates, but it is also large at any steep smooth feature, leading to overcoverage in non-shock steep regions. A learned MLP combining both features can disambiguate "steep and uncertain" from "steep and certain". Pilot results confirm the qualitative pattern.

6. Methodology

6.1 Pipeline overview

The end-to-end pipeline has six stages:

1. Train a PINN u_θ on the chosen PDE.
2. Compute a high-fidelity reference u^* on a dense evaluation grid.
3. Compute physics-derived features $\varphi(x)$ on the grid via autograd.
4. Split grid points into training-of-MLP, conformal-calibration, and test.
5. Fit a small MLP g_φ to predict $\log|\text{error}|$ from φ on the MLP-training split.
6. Use $s(x) = \exp(g_\varphi(\varphi(x)))$ as the nonconformity score; compute q on the conformal-calibration split; report coverage and width on test.

6.2 Feature family

Feature	Symbol	Computation	Motivation
PDE residual		R_θ	
Gradient norm	$\ \nabla u_\theta\ $	autograd, 1st derivatives	localizes sharp features
Spatial Laplacian		Δu_θ	
Hessian Frobenius	$\ \nabla^2 u_\theta\ _F$	autograd, all 2nd partials	full curvature
Log of each	$\log(\cdot + \varepsilon)$	elementwise transform	scale stabilization

For Paper 1 the core feature set is the first four. Optional extensions (TV indicator, entropy violation) are deferred to ablation studies.

6.3 The MLP score

Architecture chosen deliberately small to avoid overfitting the calibration set:

$g_phi : \mathbb{R}^k \rightarrow \mathbb{R}$ Linear($k, 32$) $\rightarrow$ ReLU Linear($32, 16$) $\rightarrow$ ReLU Linear($16, 1$) score $s(x) = \exp(g_phi(\phi(x)))$

- **Target:** $y_i = \log(|u_\theta(x_i) - u^*(x_i)| + \varepsilon)$, $\varepsilon = 1e-10$.
- **Loss:** mean squared error in log space (scale-equivariant).
- **Optimizer:** Adam, $lr = 1e-3$, weight decay = $1e-4$, 200 epochs, batch size 512.
- **Feature normalization:** standardize each feature using calibration-train mean/std.
- **Regularization:** early stopping on a 20 % sub-split of the MLP training data.

Why log-error: Errors span many orders of magnitude ($1e-1$ at the shock, $1e-4$ in smooth regions). Predicting raw error makes the MLP focus on the shock and ignore everything else. Predicting log-error gives equal weight to relative errors at all scales. This single design choice was worth $\approx 30\%$ in mean width on the Burgers pilot.

6.4 The split-of-the-split

Critical correctness point. Of the labeled set:

- **50 % $\rightarrow$ test set.** Used only for final evaluation. Never seen by either the MLP or the CP procedure.
- **50 % $\rightarrow$ calibration set.** Further split:
 - **40 % $\rightarrow$ MLP training ($D_{\text{mlp-train}}$).** Used to fit g_φ .
 - **10 % $\rightarrow$ conformal calibration (D_{conf}).** Used to compute q .

If the MLP and the CP quantile share data, the score is no longer exchangeable with test points and the marginal coverage guarantee can break. The disjoint split fixes this.

7. Models and Architectures

7.1 PINN architecture (every PDE)

Component	Specification
Type	feedforward MLP
Inputs	(t, x) for time-dependent; x for stationary
Hidden layers	6
Hidden width	50 per layer
Activation	tanh
Output	scalar $u_\theta(x)$
Initialization	Xavier/Glorot
Loss	$L_{PDE} + 10 \cdot L_{IC} + 10 \cdot L_{BC}$
Collocation	5000 points, resampled per iteration
Training schedule	Adam 10000 (lr=1e-3) $\rightarrow$ Adam 2000 (lr=1e-4) $\rightarrow$ L-BFGS 1000
Convergence target	PDE loss $\leq 1e-3$

7.2 MC-Dropout variant

Same architecture as above, with Dropout(p=0.1) inserted after every tanh activation. Used to compute MC-dropout-CP baseline. Inference: 30 stochastic forward passes; score is the per-point std.

7.3 Ensemble baseline

5 independent PINNs trained with different random seeds (0, 1, 2, 3, 4). Score is per-point std across the ensemble.

7.4 Learned scorer (the contribution)

Architecture as in §6.3. Trained on calibration data, used as nonconformity score in CP.

7.5 Reference solvers (no learning)

PDE	Reference solver	Grid
Burgers $v=0.01/\pi$	Method of lines + LSODA (or Hopf–Cole exact)	512×51
Buckley–Leverett	Welge construction (analytical)	256×51
Allen–Cahn $\epsilon=0.01$	Pseudo-spectral, implicit time stepping	1024×101

8. Code Organization

The repository should be a Python package with one module per concern:

pinn-conformal/ |– README.md |– pyproject.toml |– conformal_pinn/ # importable library | |– init.py | |– pdes/ # one file per PDE | | |– burgers.py # PDE residual + reference solver | | |– buckley_leverett.py | |– allen_cahn.py | |– pinn/ | | |– model.py # vanilla PINN, dropout PINN | | |– train.py # Adam + L-BFGS protocol | |– checkpoints/ # saved trained PINNs | |– features/ | |– derivatives.py # autograd-based features | |– scores/ | | |– residual.py # baseline scores | | |– mc_dropout.py | | |– ensemble.py | |– learned_mlp.py # the contribution | |– conformal/ | |– split_cp.py # 5-line CP wrapper | |– eval/ | |– metrics.py # coverage, width | |– plots.py # heatmaps, calibration plots | |– experiments/ # one runnable script per phase | |– 01_train_pinn.py | |– 02_compute_features.py | |– 03_baseline_scores.py | |– 04_learned_score.py | |– 05_other_pdes.py | |– 06_ablations.py | |– tests/ # unit tests | |– test_split_cp.py | |– test_features.py– results/ | |– checkpoints/ | |– tables/ `– figures/

Software-engineering rule: PDE code, model code, feature code, score code, and CP code are all independent. You can change any one of them without touching the others. This is what lets you swap a Burgers PINN for an Allen–Cahn PINN without rewriting the score logic.

9. Experimental Protocol

9.1 Per-PDE experiment cycle

For each of {Burgers, BL, AC):

1. Train PINN to convergence (see §7.1) at seeds 0–4.
2. For each seed, compute reference solution and features on the evaluation grid.
3. For each seed, compute baseline scores ($|R|$, $|\nabla u|$, $|u_{xx}|$, $\|Hess\|$, MC-dropout, ensemble).
4. For each seed, fit the learned MLP on $D_{mlp-train}$.
5. For each seed, compute split-CP intervals using each score, report coverage and width.
6. Aggregate: mean $\pm$ std across 5 seeds for every metric.

9.2 Ablations to run

Ablation	Question answered
Feature dropout (each)	Which features carry signal? Can the score be simplified?
MLP capacity	Is the default MLP big enough? Is a bigger MLP harmful?
Calibration size	How much labeled data does the wrapper need?
Cross-seed transfer	Does the MLP fit on seed 0 transfer to seeds 1–4 of the same PDE?
Cross-PDE transfer	Does the MLP fit on Burgers transfer to BL? (preview of paper 4)
α sensitivity	Does the pattern hold at $\alpha \in \{0.05, 0.1, 0.2\}$?

10. PDE Benchmarks

10.1 Burgers ($\nu = 0.01/\pi$)

Equation: $u_t + u u_x - \nu u_{xx} = 0$ on $(t, x) \in [0, 1] \times [-1, 1]$.

IC: $u(0, x) = -\sin(\pi x)$. **BC:** $u(t, \pm 1) = 0$.

Singularity: a smooth shock develops at $x = 0$ around $t \approx 1/\pi$. The classical PINN benchmark for shock-handling.

Reference: method-of-lines with 512 spatial points + LSODA, or Hopf–Cole transform exact.

10.2 Buckley–Leverett

Equation: $u_t + f(u)_x = 0$ with fractional flow $f(u) = u^2 / (u^2 + (1-u)^2/2)$ on $[0, 1] \times [0, 1]$.

IC: $u(0, x) = 1$ if $x = 0$, else 0. **BC:** $u(t, 0) = 1$.

Singularity: a non-convex flux gives a shock followed by a rarefaction (Welge construction).

Reference: the Welge construction is analytical.

10.3 Allen–Cahn ($\epsilon = 0.01$)

Equation: $u_t = \epsilon^2 u_{xx} - u(u^2 - 1)$ on $[0, 1] \times [-1, 1]$.

IC: $u(0, x) = x^2 \cos(\pi x)$. **BC:** $u(t, -1) = u(t, 1)$, periodic.

Singularity: sharp diffuse interfaces of width $O(\epsilon)$; not a true shock but qualitatively similar UQ challenge.

Reference: pseudo-spectral method, 1024 modes, implicit Euler.

11. Baselines

Eight scores compared in the main results table:

Score	Type	Reference
	R	(residual)

Score	∇_u Type	Reference
	u_{xx}	
$\ \text{Hess}\ _F$	single-feature	ours, ablation
MC-dropout	stochastic	Yu et al. 2025
Ensemble (5 seeds)	stochastic	Lakshminarayanan et al. 2017
LCQE (localized residual)	localized CP	Yu et al. 2025
Learned MLP (proposed)	learned multi-feature	this paper

12. Metrics and Evaluation

Metric	Formula / definition	Target
Marginal coverage	$\mathbb{P}(y \in C(x))$ on test	$\geq 1 - \alpha$
Cond. coverage near shock	coverage on	$x - x_{\text{shock}}(t)$
Cond. coverage in bulk	coverage on	x
Mean width	mean $q \cdot s(x)$ over test	lower better
Mean width near shock	mean $q \cdot s(x)$ on shock region	key metric for sharpness claim
Calibration plot	predicted vs actual error quantiles	diagonal

Common pitfall: Spearman rank correlation between score and true error is often used as a quick diagnostic. Do NOT use it as the primary metric. A score with high rank correlation can produce miscalibrated CP intervals (the early Burgers pilot showed this clearly). Coverage and width are the metrics that match the deliverable.

13. Phase-by-Phase Execution Plan

The project is divided into eight phases. Each phase produces a checkpoint and ends with a mentor meeting (★).

Phase	Title	Weeks	Output
1	Setup, repo skeleton, environment	1	working repo, all imports work
2	Train and verify Burgers PINN	1	PINN checkpoint, sanity-check heatmap
3	Reference solutions and feature pipeline	1	all features computed; visualized
4	Split-CP wrapper + unit tests ★	1	tested 5-line CP module
5	Single-feature baselines on Burgers ★	1	Table 1 of paper
6	Learned MLP score on Burgers ★	2	Table 2 – the contribution
7	Buckley–Leverett and Allen–Cahn	2	secondary results
8	Ablations, multi-seed, paper writeup ★	3	submitted manuscript

★ = mentor meeting at end of phase. Total: 12 weeks of focused work.

13.1 Phase 1 — Setup (Week 1)

- Create conda env, install torch, numpy, scipy, matplotlib, pytest.
- Create repository skeleton per §8.
- Initialize git, write a one-line README.md.
- Validate: `python -c "import torch; print(torch.__version__)"` works.

13.2 Phase 2 — Train Burgers PINN (Week 2)

- Implement `conformal_pinn/pinn/model.py`: the 6-layer 50-width tanh MLP.
- Implement `conformal_pinn/pinn/train.py`: Adam $\rightarrow$ L-BFGS schedule.
- Implement `conformal_pinn/pdes/burgers.py`: PDE residual function and reference solver.
- Run `experiments/01_train_pinn.py`: train at seed 0, save checkpoint.
- Visualize: heatmap of $u_\theta(t, x)$. Should show a smooth tilted-V with a sharp transition at $x=0$.
- Verify final PDE loss $\leq 1e-3$.

13.3 Phase 3 — Features (Week 3)

- Implement `conformal_pinn/features/derivatives.py` using autograd.
- Run `experiments/02_compute_features.py`: compute all four features on the 512×51 grid.
- Plot all features as log10 heatmaps alongside the true error heatmap (6-panel figure).
- Sanity check: each feature is non-NaN, non-zero, has a sensible range.

13.4 Phase 4 — CP wrapper (Week 4) ★

- Implement `conformal_pinn/conformal/split_cp.py` per Appendix B.
- Write unit tests: with constant score, marginal coverage hits target $\pm 2\%$ at $\alpha=0.1$.
- Run pytest. All tests pass.
- Mentor checkpoint: review code, agree on baseline numbers we expect.

13.5 Phase 5 — Baselines on Burgers (Week 5) ★

- Run `experiments/03_baseline_scores.py`: produces a table of (cov_m, cov_n, cov_f, width, w_near) for the four single-feature scores plus MC-dropout and ensemble.
- Expected pattern: marginal coverage = 0.90 for all (CP guarantee); near-shock coverage UNDERCOVERS for residual and curvature scores; gradient-norm OVERCOVERS with bloated width.
- Mentor checkpoint: confirm pattern matches expectations; if not, debug before continuing.

13.6 Phase 6 — Learned score (Weeks 6–7) ★

- Implement `conformal_pinn/scores/learned_mlp.py` per Appendix B (the LearnedScore class).
- Run `experiments/04_learned_score.py`: fit MLP, compute CP intervals, report metrics.
- Compare against baselines from Phase 5. Verify $\text{cov}_n \geq 0.90$ and width comparable.
- Generate the headline figure: 3-panel heatmap (error | residual | learned score).
- Mentor checkpoint: this is the main result. Discuss whether to proceed to other PDEs.

13.7 Phase 7 — Other PDEs (Weeks 8–9)

- Implement Buckley–Leverett: PDE residual, Welge reference, train PINN, run experiments.
- Implement Allen–Cahn: PDE residual, spectral reference, train PINN, run experiments.
- Generate equivalents of Tables 1 and 2 for each PDE.
- Compile cross-PDE comparison table for paper.

13.8 Phase 8 — Ablations and writeup (Weeks 10–12) ★

- Run all ablations (§9.2). Generate ablation tables.
- Run multi-seed (5 seeds) for each PDE. Add error bars to all reported numbers.
- Write paper draft per §15.
- Mentor checkpoint: review draft; one round of revisions; submit.

14. Weekly Todos

Concrete week-by-week todos. Treat as a contract: at the end of each week you should have produced the listed artifact. If not, ask early.

Week	Todo	Artifact at end of week
1	Set up environment, repo skeleton; read sections 0-3 of this document; read papers from §18 reading list 1–3.	working repo, conda env, reading notes
2	Implement and train Burgers PINN at seed 0.	saved checkpoint; sanity-check heatmap; PDE loss $\leq 1e-3$

Week	Implement reference solver; implement feature pipeline; produce 6-panel feature plot.	features.png, showing log10 of error and four features Artifact at end of week
4	Implement split-CP wrapper; write unit tests; mentor meeting.	split_cp.py, passing tests, ★ meeting notes
5	Run all 6 single-feature/stochastic baselines; produce Table 1; mentor meeting.	Table 1 (CSV); ★ meeting notes
6	Implement learned-MLP scorer; run on Burgers; verify hits target near-shock coverage.	Table 2 row for Burgers; headline figure
7	Multi-seed (5 seeds) on Burgers; mentor meeting.	Table 2 with error bars; ★ meeting notes
8	Implement and run Buckley–Leverett pipeline.	BL Tables 1 + 2
9	Implement and run Allen–Cahn pipeline.	AC Tables 1 + 2
10	Run all ablations (§9.2); produce ablation tables.	ablation tables; final figures
11	Write paper draft (sections 1–4 first, then 5+).	draft v1 sent to mentor
12	Revisions; format for target venue; submit. Mentor checkpoint.	submitted manuscript; ★ final meeting

15. Writing the Paper

15.1 Section structure

- Abstract.** ~150 words. Problem (PINN UQ at singularities lacks calibrated guarantees), method (learned multi-feature CP score), headline result (cov_n improvement with maintained width), contribution.
- Introduction (1–1.5 pages).** Why UQ for PINNs matters; why singularities are the failure mode; one-paragraph contribution.
- Background (1 page).** PINNs (1 paragraph), conformal prediction (1 paragraph), related work (1 paragraph) explicitly comparing to Yu et al., Gopakumar et al., Kumar et al.
- Method (1.5 pages).** Feature definitions (§6.2); MLP score (§6.3); split-of-the-split (§6.4); proposition (§5.2) with proof sketch; explicit pseudocode.
- Experiments (3–4 pages).** Table 1 (single-feature baselines); Table 2 (learned, the contribution); main figure (3-panel heatmap); BL and AC tables; ablations.
- Discussion (1 page).** Limitations: requires labeled calibration data; per-PINN; assumes exchangeability; conditional coverage is empirical not theoretical.
- Conclusion (½ page).** Summary; pointer to follow-up directions (theory, inverse problems, transfer).

15.2 Writing tips

- Lead with a picture, not a formula. The first figure should be: error | residual score | learned score — three heatmaps side by side.
- State the marginal coverage proposition as a numbered theorem with half-page proof. Do NOT overclaim conditional coverage.
- Be explicit about what is novel and what is standard. CP machinery is standard; features are standard; the combination and the singularity-bearing benchmarks are novel.
- Address Yu et al. 2025 prominently. Reviewers will find that paper. Compare on PDE choice (we add shocks, they don't), score type (we learn multi-feature, they hand-pick single).
- Report 5-seed mean ± std for every number. No single-seed claims.

15.3 Target venues

Venue	Why	Timeline
Neural Networks (Elsevier)	values methodology + theory; PINNs are central there	4–8 months
J. Comput. Phys.	values numerical experiments + reference solutions	6–12 months
TMLR	open access, fast turnaround, no length limit	2–4 months
NeurIPS workshop (ML4PS)	good first-paper venue, exposure to ML+physics audience	varies

16. Risk Register

Risk	Lik.	Impact	Mitigation

PINN does not converge to $\leq 1e-3$ on BL or AC	med Lik.	high Impact	use causality-weighting; if both fail, document and proceed with available convergence
MLP overfits the calibration set	low	med	small architecture (32-16-1), weight decay, train/val split, early stopping
Cross-seed transfer fails entirely	med	low	expected per pilot v4; framing already accommodates per-PINN deployment
Yu et al.'s LCQE matches us on shocks	low	med	they don't test on shocks; if they do match, our gain shrinks but doesn't vanish
Compute budget for 3 PDEs $\times$ 5 seeds $\times$ 8 baselines	med	med	amortize: train each PINN once, score-engineer in post; CPU is enough
Reviewer asks for industrial / clinical validation	low	low	scope is methodology; not clinical claims
Reference solution for BL is wrong	low	high	Welge is analytical; cross-check with finite-volume reference

17. Troubleshooting Reference

Symptom	Likely cause	Fix
PINN PDE loss stuck at ~ 1.0	lr too high; unlucky init	drop lr to $1e-4$; try 5 different seeds
Reference solver fails (LSODA)	grid too coarse near shock	increase N to 1024; tighten tolerances
Features come out as NaN	forgot <code>requires_grad_(True)</code>	see Phase 3 code in Appendix B
MLP loss explodes	$\log(\text{error} + 1e-10)$ is huge for tiny errors	clip log-error at e.g. -15
CP marginal coverage far from 0.90	score has zeros (divide by zero)	always add $1e-10$ to s before dividing
$\text{cov}_m \approx 0.90$ but $\text{cov}_n = 0$	shock region has near-zero score	check feature scaling; use log-features as inputs
Learned score never beats	R	-CP
Cross-seed transfer always fails	expected at low convergence	report honestly as per-PINN refinement
Numbers swing wildly across runs	single seed only	always report 5-seed mean $\pm$ std

18. Reading List

Read in order. Total: $\sim 6\text{--}8$ hours, do in week 1.

1. **Raissi, Perdikaris, Karniadakis (2019)**. "Physics-Informed Neural Networks: A Deep Learning Framework for Solving Forward and Inverse Problems Involving Nonlinear PDEs." J. Comput. Phys. 378, 686–707. Read sections 1–3.
2. **Vovk, Gammerman, Shafer (2005)**. Algorithmic Learning in a Random World. Springer. Chapter 2 only — split conformal prediction.
3. **Lei, G'Sell, Rinaldo, Tibshirani, Wasserman (2018)**. "Distribution-Free Predictive Inference for Regression." JASA 113, 1094–1111. Sections 1–3.
4. **Angelopoulos & Bates (2021)**. "A Gentle Introduction to Conformal Prediction." arXiv:2107.07511. Chapters 1–4 (free PDF).
5. **Yu, Ho, Wang (2025)**. arXiv:2509.13717. Read fully — closest competitor.
6. **Gopakumar et al. (2025)**. arXiv:2502.04406. Read fully.
7. **Kumar et al. (2025)**. arXiv:2509.21955. Read for the learned-score methodology.
8. **Foygel-Barber, Candès, Ramdas, Tibshirani (2021)**. "The limits of distribution-free conditional predictive inference." Information & Inference. Read introduction and main impossibility theorem.
9. **Romano, Patterson, Candès (2019)**. "Conformalized Quantile Regression." NeurIPS. Skim.

19. Mentor Checkpoints and Communication

Five compulsory mentor meetings, marked ★ in the timeline. Every meeting:

- **Bring a one-page written summary** of what you did since last meeting, what you learned, what you decided, and what is unclear.
- **Bring all plots and tables** generated in the period.
- **Bring a list of decisions you are uncertain about.** Asking is the entire point of the meeting.

Between meetings:

- Write to the mentor within 24 hours if something doesn't match the expected output described in this document.
- Do not silently re-design. If you want to change the methodology, ask first.
- Maintain a running journal (markdown file in the repo): one entry per session, what you did, what you observed.

Most important rule: Ask early, not late. Every silent week of going down the wrong path costs the project more than every week of asking too many questions. The mentor expects questions.

Appendix A. Default Hyperparameters

A.1 PINN training

Architecture: MLP(2, 50, 50, 50, 50, 50, 50, 1) with tanh Phase 1 Adam: lr=1e-3, n_iter=10000, n_collocation=5000 (resampled per iter) Phase 2 Adam: lr=1e-4, n_iter=2000 Phase 3 L-BFGS: max_iter=1000, tolerance_grad=1e-7 Loss weights: 1.0 (PDE) : 10.0 (IC) : 10.0 (BC) Convergence target: PDE loss <= 1e-3 Random seeds: {0, 1, 2, 3, 4} for multi-seed runs

A.2 Score MLP

Architecture: Linear(k, 32) -> ReLU -> Linear(32, 16) -> ReLU -> Linear(16, 1) Default k = 4 ($|R|$, $|grad u|$, $|u_{xx}|$, $||Hess||_F$) Optimizer: Adam, lr=1e-3, weight_decay=1e-4 Epochs: 200 (with early stopping, patience 30) Batch size: 512 Train / validation split (within MLP-train): 80 / 20 Target: $\log(|err| + 1e-10)$, clipped at -15 lower bound Feature normalization: standardize on MLP-train mean/std

A.3 CP wrapper

alpha (target miscoverage): 0.1 primary; 0.05, 0.2 sensitivity Calibration size: 50% of evaluation grid (~13000 for Burgers 512x51) Test size: other 50% Within calibration: 80% MLP-train, 20% conformal-calibration Quantile rule: $\text{ceil}((n+1)(1-\alpha)) / n$ empirical quantile Region masks (Burgers): near = $|x| < 0.1$; far = $|x| > 0.4$

A.4 Reference solvers

Burgers ($\nu = 0.01/\pi$): Method of lines, N = 512 spatial, n_t = 51 time snapshots Solver: `scipy.integrate.solve_ivp(method='LSODA', rtol=1e-8, atol=1e-10)` Cross-check: Hopf-Cole exact (analytical)

Buckley-Leverett: Welge construction (analytical, no solver needed) Cross-check: 5th-order WENO finite volume on N = 1024

Allen-Cahn ($\epsilon = 0.01$): Pseudo-spectral with N = 1024 modes Implicit Euler for stiff term, dt = 1e-4

Appendix B. Code Skeletons

B.1 PINN model and training

```
# conformal_pinn/pinn/model.py
import torch
import torch.nn as nn

class PINN(nn.Module):
    def __init__(self, in_dim=2, hidden=50, depth=6):
        super().__init__()
        layers = [nn.Linear(in_dim, hidden), nn.Tanh()]
        for _ in range(depth - 1):
            layers += [nn.Linear(hidden, hidden), nn.Tanh()]
        layers += [nn.Linear(hidden, 1)]
        self.net = nn.Sequential(*layers)
    def forward(self, t, x):
        return self.net(torch.cat([t, x], dim=-1))

# conformal_pinn/pinn/train.py
def train_pinn(model, pde, n_iter_adam=10000, n_iter_lbfgs=1000,
               n_col=5000, seed=0):
    """pde object provides: pde.residual(model, t, x), pde.ic(t, x), pde.bc(...)."""
    torch.manual_seed(seed)
    opt = torch.optim.Adam(model.parameters(), lr=1e-3)
    for it in range(n_iter_adam):
```

```

t_c = torch.rand(n_col, 1, requires_grad=True)
x_c = pde.sample_x(n_col).requires_grad_(True)
opt.zero_grad()
L_pde = pde.residual_loss(model, t_c, x_c)
L_ic = pde.ic_loss(model)
L_bc = pde.bc_loss(model)
loss = L_pde + 10*L_ic + 10*L_bc
loss.backward(); opt.step()
if (it+1) % 1000 == 0:
    print(f"adam {it+1}: pde={L_pde.item():.3e} ic={L_ic.item():.3e} bc={L_bc.item():.3e}")
# L-BFGS finetune
lbfgs = torch.optim.LBFGS(model.parameters(), max_iter=n_iter_lbfgs,
                           tolerance_grad=1e-7, history_size=50)

def closure():
    lbfgs.zero_grad()
    # ... same loss, on a fixed large collocation batch ...
    loss.backward()
    return loss
lbfgs.step(closure)
return model

```

B.2 Feature pipeline

B.2 Feature pipeline

```

python
# conformal_pinn/features/derivatives.py
import torch

def compute_features(model, t_pts, x_pts, nu):
    """t_pts, x_pts: (N, 1) tensors. Returns dict of (N,) numpy arrays."""
    t = t_pts.detach().requires_grad_(True)
    x = x_pts.detach().requires_grad_(True)
    u = model(t, x)
    u_t = torch.autograd.grad(u, t, torch.ones_like(u), create_graph=True)[0]
    u_x = torch.autograd.grad(u, x, torch.ones_like(u), create_graph=True)[0]
    u_xx = torch.autograd.grad(u_x, x, torch.ones_like(u_x), create_graph=True)[0]
    u_tt = torch.autograd.grad(u_t, t, torch.ones_like(u_t), create_graph=True)[0]
    u_tx = torch.autograd.grad(u_t, x, torch.ones_like(u_t), create_graph=True)[0]
    R = u_t + u*u_x - nu*u_xx
    g = torch.sqrt(u_t**2 + u_x**2)
    H = torch.sqrt(u_tt**2 + 2*u_tx**2 + u_xx**2)
    return {
        "u": u.detach().numpy().flatten(),
        "R": R.detach().numpy().flatten(),
        "grad_norm": g.detach().numpy().flatten(),
        "u_xx": u_xx.detach().numpy().flatten(),
        "hess_frob": H.detach().numpy().flatten(),
    }

```

B.3 Split-CP wrapper

```

python
# conformal_pinn/conformal/split_cp.py
import numpy as np

def split_cp_quantile(score_cal, err_cal, alpha=0.1):
    s = np.asarray(score_cal) + 1e-10
    r = np.asarray(err_cal) / s
    n = len(r)
    k = int(np.ceil((n + 1) * (1 - alpha)))
    k = min(k, n)
    return float(np.sort(r)[k - 1])

def cp_intervals(y_pred, score_test, q):
    half = q * (np.asarray(score_test) + 1e-10)
    return y_pred - half, y_pred + half

```

```

def coverage(y_true, lower, upper):
    return float(np.mean((y_true >= lower) & (y_true <= upper)))

def mean_width(lower, upper):
    return float(np.mean(upper - lower))

```

B.4 Learned MLP scorer

python

```

# conformal_pinn/scores/learned_mlp.py
import torch
import torch.nn as nn
import numpy as np

class LearnedScore(nn.Module):
    def __init__(self, k_features=4):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(k_features, 32), nn.ReLU(),
            nn.Linear(32, 16), nn.ReLU(),
            nn.Linear(16, 1),
        )
        self.mu = None; self.sd = None
    def fit(self, X, err, n_epochs=200, lr=1e-3, weight_decay=1e-4,
            batch_size=512, val_frac=0.2, seed=0, log_clip=-15.0):
        rng = np.random.default_rng(seed)
        n = len(err)
        idx = rng.permutation(n)
        n_val = int(val_frac * n); va = idx[:n_val]; tr = idx[n_val:]
        self.mu = X[tr].mean(axis=0); self.sd = X[tr].std(axis=0) + 1e-12
        Xtr = (X[tr] - self.mu) / self.sd
        Xva = (X[va] - self.mu) / self.sd
        ytr = np.clip(np.log(err[tr] + 1e-10), log_clip, None)
        yva = np.clip(np.log(err[va] + 1e-10), log_clip, None)
        Xtr_t = torch.tensor(Xtr, dtype=torch.float32)
        ytr_t = torch.tensor(ytr, dtype=torch.float32).unsqueeze(1)
        Xva_t = torch.tensor(Xva, dtype=torch.float32)
        yva_t = torch.tensor(yva, dtype=torch.float32).unsqueeze(1)
        opt = torch.optim.Adam(self.parameters(), lr=lr, weight_decay=weight_decay)
        best = float('inf'); patience = 30; bad = 0; best_state = None
        for ep in range(n_epochs):
            perm = np.random.permutation(len(ytr_t))
            for i in range(0, len(perm), batch_size):
                b = perm[i:i+batch_size]
                opt.zero_grad()
                loss = ((self.net(Xtr_t[b]) - ytr_t[b])**2).mean()
                loss.backward(); opt.step()
            with torch.no_grad():
                vl = ((self.net(Xva_t) - yva_t)**2).mean().item()
            if vl < best:
                best = vl
                best_state = {k: v.clone() for k, v in self.state_dict().items()}
                bad = 0
            else:
                bad += 1
                if bad > patience: break
        if best_state is not None:
            self.load_state_dict(best_state)
    def score(self, X):
        Xn = (X - self.mu) / self.sd
        with torch.no_grad():
            log_s = self.net(torch.tensor(Xn, dtype=torch.float32)).numpy().flatten()
        return np.exp(log_s) + 1e-10

```

B.5 End-to-end experiment driver

python

```

# experiments/04_learned_score.py
import torch, numpy as np
from conformal_pinn.pinn.model import PTNN

```

```

from conformal_pinn.pinn_model import PINN
from conformal_pinn.pdes.burgers import reference_solution, NU
from conformal_pinn.features.derivatives import compute_features
from conformal_pinn.scores.learned_mlp import LearnedScore
from conformal_pinn.conformal.split_cp import split_cp_quantile

```

```

model = PINN(); model.load_state_dict(torch.load('results/checkpoints/burgers_seed0.pt'))
model.eval()

```

```

t_ref, x_ref, u_ref = reference_solution()
T, X = np.meshgrid(t_ref, x_ref, indexing='ij')
t_pts = torch.tensor(T.flatten()[:, None], dtype=torch.float32)
x_pts = torch.tensor(X.flatten()[:, None], dtype=torch.float32)
feats = compute_features(model, t_pts, x_pts, NU)
err = np.abs(feats["u"] - u_ref.T.flatten())

X_feat = np.stack([np.abs(feats["R"]), feats["grad_norm"],
                        np.abs(feats["u_xx"]), feats["hess_frob"]], axis=1)

```

```

rng = np.random.default_rng(42)
N = err.size
perm = rng.permutation(N)
cal = perm[:N//2]; test = perm[N//2:]
n_cal = len(cal)
mlp_train = cal[: int(0.8 * n_cal)]
mlp_conf = cal[int(0.8 * n_cal):]

scorer = LearnedScore(k_features=4)
scorer.fit(X_feat[mlp_train], err[mlp_train])

s_conf = scorer.score(X_feat[mlp_conf])
s_test = scorer.score(X_feat[test])
q = split_cp_quantile(s_conf, err[mlp_conf], alpha=0.1)

half = q * s_test
e_test = err[test]
X_test_x = X.flatten()[test]
near = np.abs(X_test_x) < 0.1; far = np.abs(X_test_x) > 0.4

```

```

print("learned MLP score:")
print(f" cov_m = {(e_test <= half).mean():.3f}")
print(f" cov_n = {(e_test[near] <= half[near]).mean():.3f}")
print(f" cov_f = {(e_test[far] <= half[far]).mean():.3f}")
print(f" width_m = {half.mean():.3f}")
print(f" width_n = {half[near].mean():.3f}")

```

Appendix C. References

- Angelopoulos, A. N., & Bates, S. (2021). A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification. *Journal of Machine Learning Research*, 22, 1-62.
- Foygel-Barber, R., Candès, E. J., Ramdas, A., & Tibshirani, R. J. (2021). The limits of distribution-free conditional predictive inference. *Journal of the American Statistical Association*, 116(535), 1031-1042.
- Gopakumar, V., et al. (2025). Calibrated Physics-Informed Uncertainty Quantification. *arXiv:2502.04406*.
- Harris, T. A., & Liu, Y. (2025). Locally Adaptive Conformal Inference for Operator Models. *arXiv:2507.20975*.
- Kumar, D., Tayebati, S., Migliarba, F., Krishnan, R., & Trivedi, A. R. (2025). Learnable Conformal Prediction with Context-Aware Non-Exchangeable Data. *arXiv:2507.10000*.
- Lakshminarayanan, B., Pritzel, A., & Blundell, C. (2017). Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles. *International Conference on Machine Learning*, 113-123.
- Lei, J., G'Sell, M., Rinaldo, A., Tibshirani, R. J., & Wasserman, L. (2018). Distribution-Free Predictive Inference for Regression. *Journal of the American Statistical Association*, 113(523), 1064-1076.
- Liu, S., et al. (2025). On the inconsistency of strong-form residual loss for PINNs at entropy solutions. (Coupled-Integral PINN line). *arXiv:2507.10000*.
- Podina, L., et al. (2024). Conformal Physics-Informed Neural Networks for Forward and Inverse Problems (C-PINN). *arXiv:2406.10000*.
- Raissi, M., Perdikaris, P., & Karniadakis, G. E. (2019). Physics-Informed Neural Networks: A Deep Learning Framework for Solving Forward and Inverse Problems. *Journal of Computational Physics*, 378, 686-705.

Romano, Y., Patterson, E., & Candès, E. J. (2019). Conformalized Quantile Regression. NeurIPS.

Vovk, V., Gammerman, A., & Shafer, G. (2005). Algorithmic Learning in a Random World. Springer.

Yang, L., Meng, X., & Karniadakis, G. E. (2021). B-PINNs: Bayesian Physics-Informed Neural Networks. J. Comput. Phys., 425, 109913.

Yu, Y., Ho, C. H., & Wang, Y. (2025). A Conformal Prediction Framework for Uncertainty Quantification in Physics-Informed Neural Net

